

LeetCode 630: Course Schedule III

FLIPKART

GREEDY

MAX HEAP

Problem Insight

Given an array of courses where `courses[i] = [durationi, deadlinei]`. You cannot take two courses simultaneously. The goal is to return the **maximum number of courses** one can take.

Why Line Sweep Fails Here:

Line sweep is typically used for fixed intervals. Here, intervals are not fixed. For example, if a course has `duration = 100` and `deadline = 200`, it can be scheduled from $1 \rightarrow 100$, $50 \rightarrow 150$, or $100 \rightarrow 200$. As long as it finishes before the deadline, it is valid.

Algorithm & Intuition

The optimal approach combines a **Greedy Strategy with a Max Heap (Priority Queue)** to dynamically adjust our chosen courses and stay within deadlines.

Step 1: Sort by Deadline

Sort the courses primarily by their deadline (last day) in ascending order. This greedy choice ensures we always attempt to complete courses with earlier deadlines first.

Example Scenario: (5, 5), (4, 6), (2, 6)

Step 2: Try to Accept Every Course

Iterate through the sorted courses, accumulating the total time taken. For each course, add its duration to the `currtime` and push the duration into a Max Heap.

- **Take (5, 5):** Current time becomes 5 . Max Heap: [5] .
- **Take (4, 6):** Current time becomes $5 + 4 = 9$. Since $9 > 6$ (deadline), we have exceeded the valid limit.
- **Greedy Fix:** Drop the previously accepted course that took the highest amount of time. This frees up the maximum possible time while only sacrificing one course. We pop the top of the Max Heap (which is 5).
- Current time becomes $9 - 5 = 4$. Max Heap: [4] .
- **Take (2, 6):** Current time becomes $4 + 2 = 6$. Since $6 \leq 6$ (deadline), it's perfectly valid. Max Heap: [4, 2] .

Result: 2 courses are successfully scheduled.

Complexity Analysis

Time Complexity: $O(N \log N)$

Space Complexity: $O(N)$

C++ Implementation

```
class Solution {
public:
    int scheduleCourse(vector<vector<int>>& courses) {
        int n=courses.size();
        int ans=0;
        sort(courses.begin(),courses.end(),[](vector<int>& a,vector<int>& b){
            return a[1]<b[1];
        });
        priority_queue<int> maxh;
        int currtime=0;
        for(auto& course:courses){
            int dur=course[0];
            int ded=course[1];
            currtime+=dur;
            maxh.push(dur);
            ans++;
            if(currtime>ded){
                currtime-=maxh.top();
                maxh.pop();
                ans--;
            }
        }
        return ans;
    }
};
```